Charu C. Aggarwal
IBM T J Watson Research Center
Yorktown Heights, NY

Neural Network Training [Initialization, Preprocessing, Mini-Batching, Tuning, and Other Black Art]

Neural Networks and Deep Learning, Springer, 2018
Chapter 3, Section 3.3

How to Check Correctness of Backpropagation

- Consider a particular weight w of a randomly selected edge in the network.

- Let $L(w)$ be the current value of the loss.

- The weight of this edge is perturbed by adding a small amount $\epsilon > 0$ to it.

- Estimate of derivative:

$$\frac{\partial L(w)}{\partial w} \approx \frac{L(w+\epsilon) - L(w)}{\epsilon} \tag{12}$$

- When the partial derivatives do not match closely enough, it might be indicative of an incorrectness in implementation.

What Does "Closely Enough" Mean?

- Algorithm-determined derivative is G_e and the approximate derivative is G_a.

$$\rho = \frac{|G_e - G_a|}{|G_e + G_a|} \tag{13}$$

- The ratio should be less than 10^{-6}.

- If ReLU is used, the ratio should be less than 10^{-3}.

- Should perform the checks for a sample of the weights a few times during training.

Stochastic Gradient Descent

- We have always worked with *point-wise* loss functions so far.
  - Corresponds to stochastic gradient descent.
  - In practice, stochastic gradient descent is only a randomized approximation of the true loss function.
- True loss function is typically additive over points.
  - **Example:** Sum-of-squared errors in regression.
  - Computing gradient over a single point is like sampled gradient estimate.

Mini-batch Stochastic Gradient Descent

- One can improve accuracy of gradient computation by using a batch of instances.
  - Instead of holding a vector of activations, we hold a matrix of activations in each layer.
  - Matrix-to-matrix multiplications required for forward and backward propagation.
  - Increases the memory requirements.
- Typical sizes are powers of 2 like 32, 64, 128, 256

Why Does Mini-Batching Work?

- At early learning stages, the weight vectors are very poor.
  - Training data is highly redundant in terms of important patterns.
  - Small batch sizes gives the correct direction of gradient.
- At later learning stages, the gradient direction becomes less accurate.
  - But some amount of noise helps avoid overfitting anyway!
- Performance on out-of-sample data does not deteriorate!

Feature Normalization

- **Standardization:** Normalize to zero mean and unit variance.

- **Whitening:** Transform the data to a de-correlated axis system with principal component analysis (mean-centered SVD).
  - Truncate directions with extremely low variance.
  - Standardize the other directions.

- **Basic principle:** Assume that data is generated from Gaussian distribution and give equal importance to all directions.

Weight Initialization

- Initializations are surprisingly important.
  - Poor initializations can lead to bad convergence behavior.
  - Instability across different layers (vanishing and exploding gradients).
- More sophisticated initializations such as pretraining covered in later lecture.
- Even some simple rules in initialization can help in conditioning.

Symmetry Breaking

- Bad idea to initialize weights to the same value.
  - Results in weights being updated in lockstep.
  - Creates redundant features.
- Initializing weights to random values breaks symmetry.
- Average magnitude of the random variables is important for stability.

Sensitivity to Number of Inputs

- More inputs increase output sensitivity to the average weight.
  - Additive effect of multiple inputs: variance linearly increases with number of inputs r.
  - Standard deviation scales with the square-root of number of inputs r.
- Each weight is initialized from Gaussian distribution with standard deviation $\sqrt{1/r}$ ($\sqrt{2/r}$ for ReLU).
- **More sophisticated:** Use standard deviation of $\sqrt{2/(r_{in} + r_{out})}$.

Tuning Hyperparameters

- Hyperparameters represent the parameters like number of layers, nodes per layer, learning rate, and regularization parameter.

- Use separate validation set for tuning.

- Do not use same data set for backpropagation training as tuning.

Grid Search

- Perform grid search over parameter space.
  - Select set of values for each parameter in some "reasonable" range.
  - Test over all combination of values.
- Careful about parameters at borders of selected range.
- **Optimization:** Search over coarse grid first, and then drill down into region of interest with finer grids.

How to Select Values for Each Parameter

- Natural approach is to select uniformly distributed values of parameters.
  - Not the best approach in many cases! $\Rightarrow$ Log-uniform intervals.
  - Search uniformly in reasonable values of log-values and then exponentiate.
  - **Example:** Uniformly sample log-learning rate between -3 and -1, and then raise it to the power of 10.

Sampling versus Grid Search

- With a large number of parameters, grid search is still expensive.
- With 10 parameters, choosing just 3 values for each parameter leads to $3^{10} = 59049$ possibilities.
- Flexible choice is to sample over grid space.
- Used more commonly in large-scale settings with good results.